

Foundations of Reinforcement Learning

Wenya Xie

1 Optimization Objective in Reinforcement Learning

The agent generates actions based on a policy $\pi_\theta(a|s)$, and the environment returns a reward r_t . The goal is to **maximize the expected return**:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[R]$$

Intuitively, the agent should increase the probability of actions that lead to high rewards.

2 Mathematical Foundation

Starting from $J(\theta) = \mathbb{E}_{\pi_\theta}[R]$, we can express it as an integral over all trajectories τ :

$$J(\theta) = \int P_\theta(\tau) R(\tau) d\tau.$$

Taking the gradient and applying the log-derivative trick:

$$\nabla_\theta J(\theta) = \int P_\theta(\tau) R(\tau) \nabla_\theta \log P_\theta(\tau) d\tau = \mathbb{E}_{\tau \sim P_\theta}[R(\tau) \nabla_\theta \log P_\theta(\tau)].$$

Since $P_\theta(\tau) = p(s_1) \prod_t \pi_\theta(a_t|s_t) p(s_{t+1}|s_t, a_t)$, only π_θ depends on θ :

$$\nabla_\theta \log P_\theta(\tau) = \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t|s_t).$$

Thus the **policy gradient** becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=1}^T R(\tau) \nabla_\theta \log \pi_\theta(a_t|s_t) \right].$$

3 Evolution of RL Algorithms

3.1 REINFORCE

The simplest policy gradient method:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[R \nabla_\theta \log \pi_\theta(a|s)].$$

However, R only reflects absolute performance. Without a baseline, gradient variance is large and training is unstable.

3.2 Variance Reduction and the Actor–Critic Framework

In REINFORCE, the policy gradient depends directly on the raw return:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [R_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)].$$

Since R_t fluctuates greatly due to stochastic rewards and long-term uncertainty, the gradient estimate suffers from high variance and unstable updates.

A simple yet effective solution is to **subtract a baseline** $b(s_t)$:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [(R_t - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)].$$

This keeps the expected gradient unchanged because $\mathbb{E}_{\pi_{\theta}} [b(s_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] = 0$, but it **reduces variance by removing the predictable part of R_t** .

The optimal baseline is the expected return given the current state, $V(s_t) = \mathbb{E}[R_t | s_t]$. Learning $V(s_t)$ with a separate network yields the **actor–critic framework**:

$$A_t = R_t - V(s_t), \quad \nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)].$$

Intuitively, we introduce a baseline $V(s_t)$ representing the expected reward from this state over all possible trajectories. By comparing the actual return with this expectation, the agent can decide whether to increase or decrease the probability of taking the current action. **This simple adjustment makes training more stable without changing the true optimization objective.**

3.3 PPO (Proximal Policy Optimization)

To make better use of data collected from the previous policy, PPO adopts **importance sampling** to evaluate the new policy using old trajectories:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}.$$

The idea of importance sampling is to **reuse data sampled from the old policy** while still computing an unbiased estimate of how the new policy would perform. In other words, it allows us to update the new policy **using trajectories generated under the old one**.

The objective is then written as:

$$L^{PG} = \mathbb{E}_{t \sim \pi_{\text{old}}} [r_t(\theta) A_t].$$

However, if the new policy deviates too much from the old one, **the old trajectories no longer represent the new policy’s behavior well**, making the importance weights $r_t(\theta)$ unreliable and leading to unstable or biased updates.

To prevent such large deviations, PPO introduces a **clipping mechanism** that limits the policy update:

$$L^{CLIP} = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)].$$

This clipping ensures that the new policy stays close to the old one, allowing learning to proceed in small, reliable steps. In short, PPO **keeps the update direction** of policy gradients but **restricts its magnitude**, achieving a **balance between stability and learning efficiency**.

3.4 RLHF (Reinforcement Learning from Human Feedback)

In language generation, there is **no natural reward from the environment**. We first train a **reward model** R_ϕ on human preference data to score the quality of model responses.

During RLHF training, the policy model π_θ generates rollouts (i.e., complete responses to prompts), and the reward for each response combines the reward model’s score with a **KL penalty** that keeps the new policy close to a fixed reference model π_{ref} :

$$r = R_\phi(\text{prompt}, \text{response}) - \beta D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)).$$

This KL regularization prevents the policy from drifting too far from human-like behavior while still improving responses according to human preference.

The optimization objective remains PPO-style:

$$L^{\text{RLHF}}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)],$$

where the reward r above replaces the standard environment reward. **In essence, RLHF can be viewed as PPO with an additional KL regularization term in the reward.**

3.5 GRPO (Generalized Reward-Policy Optimization)

In RLHF, training a separate value network (critic) $V(s)$ to estimate advantages is often **unstable and computationally expensive**, since each state corresponds to a long text sequence and requires an additional forward pass. To simplify training, **GRPO removes the critic** and directly normalizes rewards within each batch to compute relative advantages:

$$A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}.$$

This batch normalization preserves the relative ordering of samples and thus maintains a similar gradient direction to PPO, while **avoiding critic fitting and value bootstrapping**.

In practice, GRPO keeps the same PPO-style objective:

$$L^{\text{GRPO}}(\theta) = \mathbb{E}_i[\min(r_i(\theta)A_i, \text{clip}(r_i(\theta), 1 - \epsilon, 1 + \epsilon)A_i)],$$

but replaces the estimated advantage $A_t = R_t - V(s_t)$ with the normalized batch advantage above. This makes training simpler, cheaper, and more stable for large-scale RLHF setups.